DIGGIBYTE

CREATING VALUE WITH DATA

DIGGIBYTE
CREATING VALUE WITH DATA

SPARK & PYSPARK

DIGGIBYTE
CREATING VALUE WITH DATA

Agenda

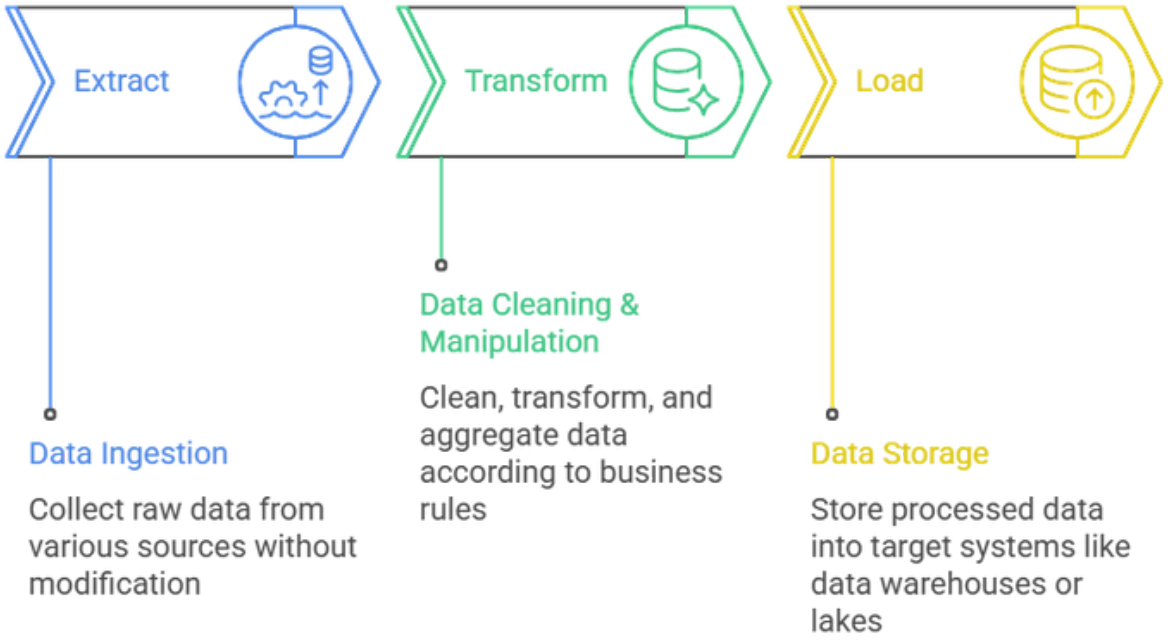
PYSPARK

- ETL
- Loads - Types
- Tables
- Libraries

DIGGIBYTE
CREATING VALUE WITH DATA

ETL – Extract, Transform, Load

ETL is a data integration process that:



Example:

Extract

```
df = spark.read.csv("data.csv", header=True, inferSchema=True)
```

Transform

```
df_clean = df.filter(df.age > 18).dropDuplicates()
```

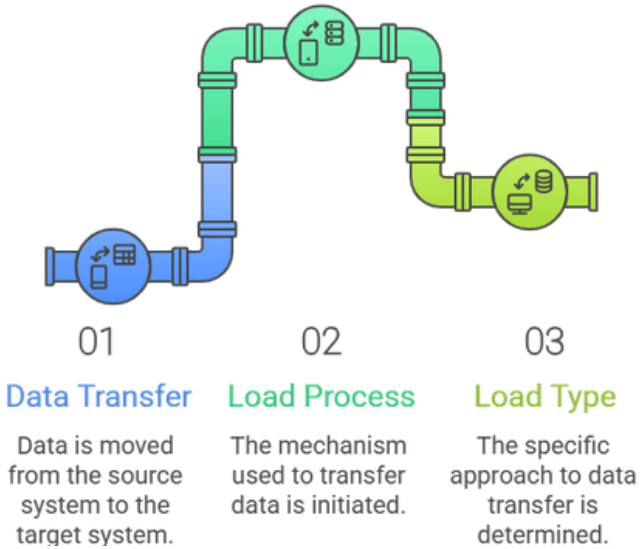
Load

```
df_clean.write.mode("overwrite").parquet("output/")
```

DIGGIBYTE
CREATING VALUE WITH DATA

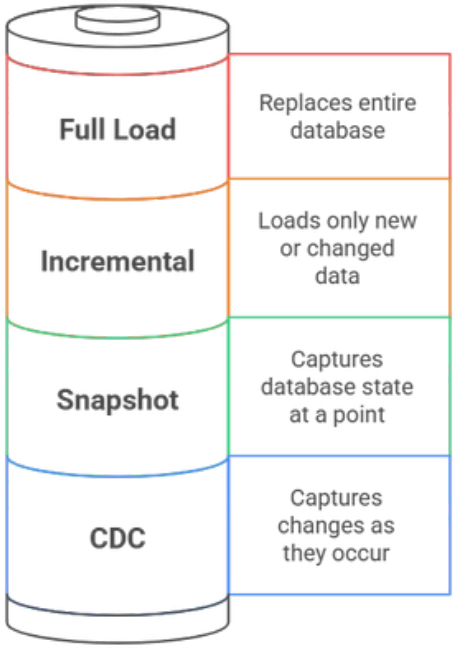
What is a Load in Data Engineering?

Process of moving data from Source → Target




```
df = spark.read.parquet("source/")
df.write.mode("overwrite").parquet("target/")
```

Types of Loads



DIGGIBYTE
CREATING VALUE WITH DATA

Full Load

- This replaces the entire target table with new data every time.
- Old data is deleted and new data is loaded completely.

Example:

```
df.write.mode("overwrite").parquet("target/")
```

id	name	city
1	Ravi	Chennai
2	Anu	Madurai

→

id	name	city
1	Ravi	Bangalore
2	Anu	Madurai

Advantages

- Simple implementation
- No complex logic

Disadvantages

- Time-consuming for large data
- Higher storage and processing cost

DIGGIBYTE
CREATING VALUE WITH DATA

Incremental Load

- Only new or updated records are loaded into the target system.
- Existing data remains unchanged.

Example:

```
last_load_time = "2026-03-01"
df_new = df.filter(df.last_updated > last_load_time)
df_new.write.mode("append").parquet("target/")
```

When to Use?

- Large datasets
- Daily or hourly updates
- Production systems

Cons:

- Requires tracking column timestamp or ID
- Slightly complex logic

id	name	city
1	Ravi	Chennai
2	Anu	Madurai

→

id	name	city
1	Ravi	Bangalore
1	Anu	Bangalore

DIGGIBYTE
CREATING VALUE WITH DATA

Snapshot Loads

- Stores the entire dataset as a snapshot at a specific time.
- Each load keeps historical version.
- Each day creates new snapshot.

Example

```
df_snapshot = df.withColumn("snap_date", current_date())
df_snapshot.write.mode("append").parquet("snapshot_table/")
```

Full Snapshot

- Entire table is stored with snapshot_date
- Each day creates a new version
- Old data is never deleted

Example

snap_date	id	name	city
01/03/2026	1	Ravi	Chennai
01/03/2026	2	Anu	Madurai

snap_date	id	name	city
02/03/2026	1	Ravi	Bangalor
02/03/2026	2	Anu	Madurai

DIGGIBYTE
CREATING VALUE WITH DATA

Incremental Snapshot

When data changes:

- Old record is marked as inactive (current = N)
- End date is updated
- New record inserted with current = Y
- Maintains complete history of changes

id	name	city	strt_date	end_date	current
1	Ravi	Vizag	01/03/20...	02/03/2...	Y - > N

id	name	city	strt_date	end_date	current
1	Ravi	Bglre	02/03/2...	NULL	Y

Pros: Eff storage, Tracks historical changes

Cons: complex logic, date/version tracking

Rolling Snapshot

- Stores data only for a fixed time window (e.g : last 3 days)
- Oldest snapshot is removed automatically
- Keeps recent performance data only

ss_date	total_sales
01/03/2026	10,000
02/03/2026	12,000
03/03/2026	11,500

ss_date	total_sales
02/03/2026	12,000
03/03/2026	11,500
04/03/2026	13,000

On 04-03-26
(Rolling Applied):

Pros : Controls storage growth ,Faster querying

Cons : Old historical data is lost.

DIGGIBYTE
CREATING VALUE WITH DATA

Change Data Capture (CDC)

- Data loading technique that captures only the changes (Insert, Update, Delete) made to the source data and applies them to the target system.
- Instead of reloading the entire table, CDC processes only modified records.

Example:

id	name	city
1	Ravi	Chennai
2	Anu	Madurai

→

id	name	city
1	Ravi	Bangalore
2	Anu	Madurai
3	Kiran	Salem

→

id	operation	city
1	update	Bangalore
3	insert	salem

```
df_changes = df.filter(df.operation.isin("INSERT", "UPDATE", "DELETE"))
df_changes.write.mode("append").parquet("cdc_table/")
```

DIGGIBYTE
CREATING VALUE WITH DATA

Timestamp-Based CDC

Uses a column like last_updated.

Logic:

- Select rows where **last_updated > last_load_time.**

Cons: Cannot easily detect deletes

```
last_load_time = "2026-03-01"
df_changes = df.filter(df.last_updated > last_load_time)
```

Log-Based CDC

Reads database transaction logs directly.

Captures:

- Insert logs
- Update logs
- Delete logs

Pros: No performance impact on source tables , Real-time capture possible

Cons: More complex setup

Trigger-Based CDC

Database triggers capture changes and store them in a change table.

When:

- INSERT
- UPDATE
- DELETE

Trigger writes operation into CDC table.

Pros : Detects all operations

Cons : Adds load to source database

Timestamp-Based CDC

Uses a column like last_updated.
Logic:

- Select rows where **last_updated > last_load_time.**

```
last_load_time = "2026-03-01"
df_changes = df.filter(df.last_updated > last_load_time)
```

Log-Based CDC

Reads database transaction logs directly.
Captures:

- Insert logs
- Update logs
- Delete logs

Trigger-Based CDC

Database triggers capture changes and store them in a change table.
When:

- INSERT
- UPDATE
- DELETE

Trigger writes operation into CDC table.
Pros : Detects all operations
Cons : Adds load to source database

DIGGIBYTE
CREATING VALUE WITH DATA

Tables in Spark

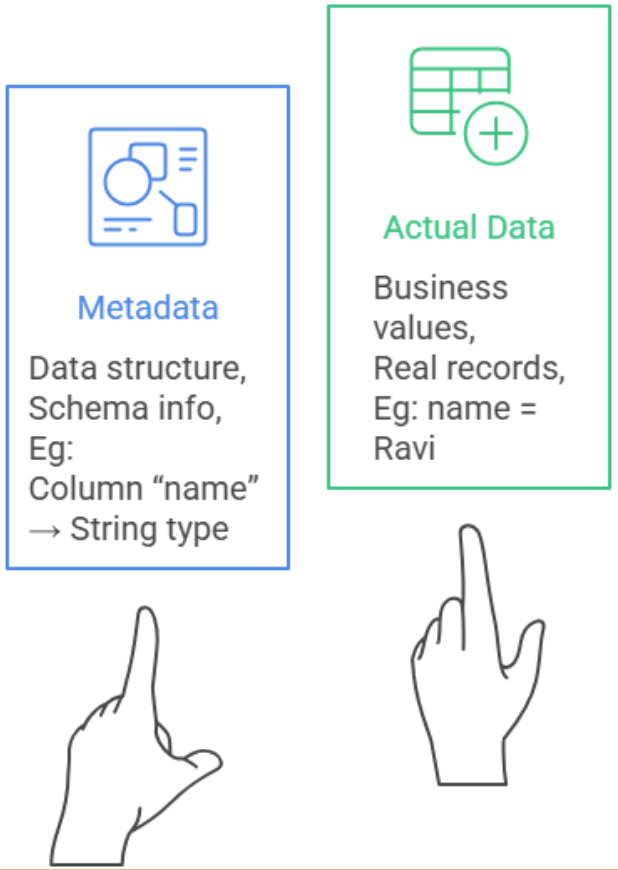
A table in Spark SQL is a structured dataset stored with:

- Table Name
- Schema (Column names & Data types)
- Storage Location

Table creation syntax.

```
df.write.saveAsTable("employees")
```

Metadata and Actual data:



DIGGIBYTE
CREATING VALUE WITH DATA

Managed Table

This is fully controlled by Spark.

Spark manages:

- Metadata
- Storage location
- Data lifecycle

```
df.write.saveAsTable("emp_managed")
```

```
DROP TABLE emp_managed;
```

- Metadata deleted
- Data folder deleted

```
Project Folder
│
├── metastore_db/
│
├── spark-warehouse/
│       └── emp_managed/
│               ├── part-00000
│               ├── part-00001
│               ├── _SUCCESS
```

DIGGIBYTE
CREATING VALUE WITH DATA

External Table

- Spark manages only metadata.
- Data location is controlled by user.

How It Works

```
df.write \
 .option("path", "C:/data/emp") \
 .saveAsTable("emp_external")
```

Now:

- Metadata → metastore
- Data → C:/data/emp

```
DROP TABLE emp_external;
```

➡ Metadata deleted
➡ Data remains safe

Differentiation :

Feature	Managed	External
Data Location	spark-warehouse	User defined
Metadata Stored	Yes	Yes
DROP deletes data?	Yes	No
Production Usage	Rare	Very Common
Data Control	Spark	User

DIGGIBYTE
CREATING VALUE WITH DATA

Libraries

A library is a collection of pre-written code

Libraries improve:

- Productivity
- Code quality
- Maintainability
- Standardization

Libraries Covered:

Cookiecutter
Generates project structures from templates

Pydantic
Validates data with Python type hints

Black
Formats code automatically

Tox
Automates testing across environments

DIGGIBYTE
CREATING VALUE WITH DATA

Cookiecutter

It is a project template generator.

In companies:

- Projects must follow architecture standards
- Folder organization must be consistent
- CI/CD pipelines expect structured layout

Advantages:

- Saves time
- Enforces standard structure
- Useful for production-grade ETL projects

Installation

pip install cookiecutter

Usage

cookiecutter https://github.com/drivendata/cookiecutter-data-science

```
my_spark_project/
│
├── src/
│   ├── transformations/
│   ├── jobs/
│   ├── utils/
│
├── tests/
├── config/
├── data/
├── requirements.txt
```

DIGGIBYTE
CREATING VALUE WITH DATA

Pydantic

This is used for data validation and schema enforcement using type hints.

Core Purpose

- Validates input data types
- Enforces schema rules
- Performs automatic type conversion
- Raises errors for invalid data

Installation pip install pydantic

Example Concept

batch_size = "25"

batch_size: int

Pydantic:

"25" → 25

Raises error if invalid ("abc")

Work Flow

Made with Napkin

DIGGIBYTE
CREATING VALUE WITH DATA

Black

Black is an automatic code formatter for Python.

- It reformats code to follow:
- PEP 8 standards
- Consistent indentation
- Proper spacing

What Black Does Internally

1. Reads .py
2. Parses syntax tree
3. Reformats acc to its style rules
4. Rewrites file

Example :

```
def calculate_total(price, tax, discount=0):
    total=price+tax-discount
    if(total>1000): print("High value order")
    return   total
```

black .

```
def calculate_total(price, tax, discount=0):
    total = price + tax - discount
    if total > 1000:
        print("High value order")
    return total
```

DIGGIBYTE
CREATING VALUE WITH DATA

Tox

Tox is a Python automation tool used to run:

- Unit tests
- Lint checks
- Code formatting checks
- Multi-version testing

In isolated virtual environments.

Basic Installation pip install tox

Basic Command tox

Work Flow:

tox.ini

```
[tox]
envlist = py, lint, format

[testenv]
deps = pytest
commands = pytest

[testenv:lint]
deps = flake8
commands = flake8 .

[testenv:format]
deps = black
commands = black --check .
```

```
my project/
  ├── app.py
  ├── tests/
  ├── tox.ini
```

Run `tox` → Create Virtual Environments → Install Dependencies → Run Commands → Any Step Fails? → Tox Fails / Tox

DIGGIBYTE
CREATING VALUE WITH DATA

QUESTIONS?

DIGGIBYTE
CREATING VALUE WITH DATA

THANK YOU